

Building Without Bytes

High-Velocity Image Composition at Scale

These slides were not exported from an app — they were **built by Bazel**, from the same workspace that builds the containers they describe.

Sponsored by **Tweag by Modus Create**

BazelCon 2026 · 9:00–10:30 · Rafael Lima

This slide is a build artifact

`bazel build //slides:build` → this deck.
`bazel build //samples/python:image` → the image we'll discuss.

Same graph. Same cache. Same guarantees.

What you build is byte-for-byte what runs in production —
starting with the deck you're watching.

Module 0 · Why: correct & hermetic

10 min · Lab: clone + `bazel build //...`

Correct means hermetic

Bazel's promise is — and *correct* means **hermetic**: a build with no external influence, so identical inputs → identical outputs.

Docker — *runtime* hermeticity

Bundles deps so the artifact runs the same everywhere. But `docker build` itself isn't hermetic: `RUN apt-get` is networked, timestamped, order-sensitive.

Bazel — *build* hermeticity

Only declared inputs are visible; identical inputs → identical, content-addressed outputs.

Docker hermetizes the **deploy** boundary; Bazel the **build**. Want both.

The "black box" build

Standard Dockerfiles are imperative scripts — and that hurts three ways:

- **Non-deterministic** — `apt-get` today ≠ `apt-get` tomorrow.
- **Coarse cache invalidation** — one early edit busts every layer below.
- **The Docker socket** — a privileged daemon: security surface + CI bottleneck.

Bazel's answer: stop **running** commands to make layers — **assemble** them from pre-built, content-addressed artifacts. The filesystem becomes data.

Module 1 · A first declarative image

15 min · Lab: build/run the image vs the Dockerfile

The "before": an imperative Dockerfile

The Docker daemon installs the `dot` system dependency with `RUN apt-get` — an opaque, stateful layer. Editing `app.py` busts everything below the `COPY`.

```
FROM python:3.12-slim-bookworm

# System dependency: the `dot` binary. Imperative, daemon-run, cache-coarse.
RUN apt-get update \
    && apt-get install -y --no-install-recommends graphviz \
    && rm -rf /var/lib/apt/lists/*

# Python dependency layer.
COPY requirements_lock.txt /tmp/requirements.txt
RUN pip install --no-cache-dir -r /tmp/requirements.txt

# App-source layer (changes here invalidate everything below in a Dockerfile).
WORKDIR /app
COPY app.py /app/app.py

ENTRYPOINT ["python", "/app/app.py", "--render", "/tmp/supply_chain.png"]
```

The payload: Python + a *non-hermetic* system dep

`graphviz` shells out to the `dot` binary — the apt dependency we must make explicit once we leave the daemon. The app renders the workshop's own graph:

```python
def build_graph() -> graphviz.Digraph:
    """The supply chain that builds this very talk."""
    dot = graphviz.Digraph("supply_chain", comment="Building Without Bytes")
    dot.attr(rankdir="LR")
    dot.attr("node", shape="box", style="rounded")

    dot.node("src", "source\n(app.py, BUILD)")
    dot.node("deps", "pip deps\n(graphviz)")
    dot.node("sysdep", "system layer\n(dot binary)", style="rounded,filled", fillcolor="lightyellow")
    dot.node("image", "oci_image")
    dot.node("slides", "slides deck")
    dot.node("registry", "registry")

    dot.edge("src", "image")
    dot.edge("deps", "image")
    dot.edge("sysdep", "image")
    dot.edge("image", "registry")
    dot.edge("src", "slides", label="transclude")  # the deck imports this file
    return dot
```

The "after": layers as declarative build outputs

No daemon. The image is composed from content-addressed layers in the action graph; changing `app.py` reuses the base + deps layers untouched.

```
oci_image(
    name = "image",
    base = "@python_base",
    entrypoint = [
        "/app/app",  # py_binary bootstrap launcher; locates /app/app.runfiles
        "--render",
        "/tmp/supply_chain.png",
    ],
    tars = [":app_layer"],
)
```

rules_oci, start to finish

1. **Pull a base** — `oci.pull` in `MODULE.bazel` (digest-pinned, multi-arch).
2. **Build a layer** — `pkg_tar` the app's runfiles (or any files).
3. **Assemble** — `oci_image(base, tars = [ ... ], entrypoint = [ ... ])`.
4. **Load / push** — `oci_load` → Docker · `oci_push` → registry.

```
oci_image(name = "image", base = "@python_base",
          tars = [":app_layer"], entrypoint = ["/app/app"])
oci_load(name = "image_load", image = ":image", repo_tags = ["bwb/app:latest"])
oci_push(name = "push", image = ":image",
         repository = "localhost:5000/bwb/app", remote_tags = ["latest"])
```

Module 2 · Hermeticity in practice

15 min · Lab: the "docker tax" benchmark

Validate images without the daemon

`container_structure_test` with `driver = "tar"` checks the image's **files and config in the Bazel sandbox** — no container runtime, no Docker daemon:

```
# Daemonless image validation (workshop Q3 / TASK §2.5).
# Run with the `tar` driver — no container runtime, no Docker daemon: the test
# inspects the image's layers and config directly in the Bazel sandbox.
schemaVersion: "2.0.0"

# File checks read the layer tarballs (no execution).
fileExistenceTests:
  - name: "py_binary launcher is present"
    path: "/app/app"
    shouldExist: true
  - name: "graphviz dep packaged in the layer"
```

```
container_structure_test(name = "image_structure_test",
    image = ":image.tar", configs = ["image_test.yaml"], driver = "tar")
```

Aside — a quick *negative* test for scripts: `docker run --network=none …` surfaces undeclared host deps (handy, but not a proof; the sandbox is).

The "docker tax"

Same suite, two ways — `py_test` (sandboxed, cached, incremental) vs. a `docker build` that pip-installs and runs pytest. Change one line, re-run:

```
bazel run //benchmark/pytest_tax:compare
#   bazel test  (warm, after edit)    ~0.8 s
#   docker build (warm, after edit)   ~2.2 s   ← per-edit "tax"
#   docker build (cold, --no-cache)  ~11   s   ← upfront "make it hermetic" cost
```

Absolute numbers are machine-dependent; the **ratio per edit** is the point — and it grows with image size and dependency count.

Module 3 · Layer optimization

25 min · Lab: layer split · dedup · distroless

Architect layers by volatility

Order layers least → most volatile so rebuilds stay cheap:

1. **Base OS** — static · 2. **Runtimes / deps** — infrequent · 3. **App code** — most volatile

Change one string in a multi-GB app:

```
$ docker build -t app .              # invalidates at COPY . /app → 2–3 min
$ bazel run //samples/rules_img:load # re-assembles one blob   → seconds
```

Bazel re-assembles only the changed layer — and uploads only that blob.

Action deduplication

Bazel's action cache keys every action on **input digests + command + env** → identical action, built **once**.

Layers are **content-addressed**, so the win compounds:

- a shared base/deps layer is **built once**, reused across *every* image;
- at the registry, an existing blob is **skipped on push**.

Optimization = make common layers **byte-identical**, and dedup does the rest.

Shared layers: two images, one blob

Both images sit on the same base + `:common_layer`; only the app layer differs.

```
oci_image(
    name = "image_a",
    base = "@python_base",
    tars = [":common_layer", ":a_layer"],  # common_layer shared with image_b ↓
)

oci_image(
    name = "image_b",
    base = "@python_base",
    tars = [":common_layer", ":b_layer"],  # same :common_layer → same blob digest
)
```

```
bazel run \
  //samples/shared_layers:digests
# SHARED (built & pushed once):
#  sha256:…  base layers
#  sha256:…  :common_layer
# + 1 unique app layer each
```

What can be shared? The **base OS** and any **byte-identical** layer (deps, config). Content-addressing makes the dedup automatic.

rules_distroless: hermetic system deps

Remember the `dot` gap? Replace `RUN apt-get install graphviz` with a **pinned manifest + lockfile** — Bazel materializes `dot` + its `.deb`s as content-addressed layers. No network at build, no daemon, no `RUN`.

```
# MODULE.bazel
apt = use_extension("@rules_distroless//apt:extensions.bzl", "apt")
apt.install(name = "graphviz_apt",
            manifest = "//samples/distroless:graphviz.yaml",
            lock = "//samples/distroless:graphviz.lock.json")
```

Bonus: **no shell, no package manager** → smaller image, smaller attack surface.

Module 4 · Push operations

15 min · Lab: push to a local registry

Daemonless push — only the changed blobs

`rules_oci` → `oci_push`; `rules_img` → `image_push`. No daemon. Both upload **only blobs the registry lacks** — the dedup from Module 3 now pays off as **bandwidth**: a shared base layer uploads once.

```
bazel run //tools:local_registry          # registry:2 on localhost:5000
bazel run //samples/rules_oci:push         # push (changed blobs only)
bazel run //samples/rules_img:push         # same image, rules_img
```

The base is pulled **multi-arch** (`linux/amd64` + `linux/arm64/v8`); building an image *index* to push both is a one-liner — no QEMU-in-Docker.

Module 5 · rules_img & shipping choices

10 min · wrap-up + Q&A

Same image, two rulesets

Same `//samples/python:app_layer` — only the ruleset differs.

`rules_oci` — `oci_image`

```
oci_image(
    name = "image",
    base = "@python_base",
    entrypoint = ["/app/app"],
    tars = ["//samples/python:app_layer"],
)
```

`rules_img` — `image_manifest` + `layer_from_tar`

```
# Reuse the exact same pkg_tar the rules_oci path uses.
layer_from_tar(
    name = "app_layer",
    src = "//samples/python:app_layer",
)

image_manifest(
    name = "image",
    base = image("python_base_img"),
    entrypoint = ["/app/app"],
    layers = [":app_layer"],
)
```

rules_img, start to finish

1. **Pull a base** — `images.pull` (rules_img's own extension) in `MODULE.bazel`.
2. **Build a layer** — `layer_from_tar` (reuse a `pkg_tar`) or `image_layer`.
3. **Assemble** — `image_manifest(base, layers = [ … ], entrypoint = [ … ])`; `image_index` for multi-arch.
4. **Load / push** — `image_load` → Docker · `image_push` → registry.

Tar-less + a Go-based incremental loader — layers stay transparent blobs until final assembly, so builds and pushes stay fast at scale.

Same artifact, three philosophies

Feature	Dockerfile	rules_oci	rules_img
Execution	Imperative	Declarative	Optimized declarative
Hermetic	No	Yes	Yes
Speed	Slow (cache miss)	Fast	Ultra-fast
Daemon req.	Yes	No	No
Tar-less	No	No	Yes

`rules_img` skips `tar`/compression and uses a Go-based incremental loader — layers stay transparent blobs until final assembly.

Takeaways

Stop scripting, start mapping — declare *what goes where*, not what to run.
Build layers, share layers — split by volatility; byte-identical layers dedupe.
Go daemonless — validate images with no runtime; push with no socket.
Evaluate `rules_img` — if `tar` is the bottleneck, upgrade.
Slide-as-Code — the deck is just another build target.

What you build is byte-for-byte what runs — layers are just build outputs.

One last loop

The Python sample renders **this repo's own dependency graph** with `dot`, and the deck imports the result — a build action drawing the graph of the build that produced this very slide.

```
bazel run //tools:render_graph \
  -- //samples/python:image
```

